GCP Master Data Engineer

6-Month Exhaustive Training & Certification Program

Advanced Architecture, Hands-on Labs, and MLOps

Abstract

This 24-week program is designed to transform you from a practitioner to a Master Data Engineer on Google Cloud. It progressively builds from core storage to advanced Data Mesh governance, integrating daily theory, weekly service-specific mini-projects, architectural use case analyses, and massive monthly Capstone projects. A tracking timetable is included at the end.

Contents

Prerequisites & How to Use This Syllabus

- **Prerequisite skills:** intermediate Python, solid SQL, Git fundamentals, Linux/CLI basics, and Docker fundamentals. You also need a Google Cloud project with billing enabled (free trial credits are sufficient for most labs).
- **Weekly cadence:** plan for roughly 10–12 hours per week — Monday to Wednesday for theory and service deep dives, Thursday for the hands-on project, and Friday for the architecture use case.
- **Deliverable expectation:** every capstone must produce a git repository, an architecture diagram, a monthly cost estimate, and a security review checklist, per the Acceptance Criteria boxes in each milestone.
- **Assessment:** the 24-week tracking timetable at the end of this document doubles as your assessment checklist — mark a week complete only when its deliverables are committed.
- **Self-checks:** every week ends with 5 self-check questions; answer them from memory before consulting the answer key in Appendix D.

Environment setup checklist (complete before Week 1):

- ☐ Install Python 3.x, Git, Docker Desktop, and VS Code (with the Python and Cloud Code extensions).
- ☐ Install the `gcloud` CLI and `bq` (bundled), plus Terraform for Week 24.
- ☐ Create a Google Cloud free-tier project with billing enabled and $300 trial credits.
- ☐ Clone a course repository structured as `data/`, `src/`, `tests/`, `infra/`, `docs/`.
- ☐ Configure credentials via environment variables and Application Default Credentials (`gcloud auth application-default login`) — never hardcode keys in source.

1 Milestone 1: Core Data Infrastructure & Storage (Month 1)

Objective: Master the foundational storage options and transactional databases.
Learning outcomes: provision and lifecycle-manage GCS buckets, deploy HA Cloud SQL instances, design globally consistent Spanner schemas, model hotspot-free Bigtable row keys

Week 1: Cloud Storage & Transfer Services

Outcomes: select storage classes from access patterns, configure lifecycle and versioning rules, execute a cross-cloud transfer job, quantify Archive-tier savings

- **Monday:** Object storage fundamentals, Storage Classes (Standard, Nearline, Coldline, Archive).
- **Tuesday:** Lifecycle Management, Versioning, and Retention Policies.
- **Wednesday:** Data Migration: Storage Transfer Service vs. Transfer Appliance.
- **Thursday:** **Hands-on Project:** Deploy a GCS bucket via CLI, configure a 30-day transition to Archive, and set up a Storage Transfer job from a public AWS S3 bucket.
- **Friday:** **Use Case & Architecture:** Design a multi-region disaster recovery storage architecture for a media streaming company serving 10PB of video assets.

Production Note: *Enable Object Versioning on buckets holding pipeline state or raw data: accidental deletes and overwrites become recoverable by restoring a noncurrent version (`gcloud storage cp gs://bucket/object#generation`). Pair with a lifecycle rule that expires old noncurrent versions so recovery ability does not silently inflate storage costs.*

Resources: Cloud Storage docs (https://cloud.google.com/storage/docs); Data Engineering Zoomcamp (https://github.com/DataTalksClub/data-engineering-zoomcamp). **Self-check:**

- Which storage class minimizes cost for data accessed less than once a year?
- What does a lifecycle rule transitioning to Archive after 30 days actually change?
- When is Transfer Appliance preferred over Storage Transfer Service?
- How does Object Versioning let you recover an accidentally deleted object?
- Why is multi-region storage not sufficient by itself for a DR plan?

Week 2: Relational Databases (Cloud SQL & AlloyDB)

Outcomes: deploy HA Cloud SQL with read replicas, execute a PITR restore drill, plan a DMS minimal-downtime cutover, identify AlloyDB columnar-engine workloads

- **Monday:** Cloud SQL Architecture, High Availability (HA), and Read Replicas.
- **Tuesday:** Database Migration Service (DMS) and Private IP connections.
- **Wednesday:** AlloyDB for PostgreSQL: Columnar engine and ML integration.
- **Thursday:** **Hands-on Project:** Deploy an HA Cloud SQL Postgres instance, create a read replica, and simulate a regional failover. DR drill: enable point-in-time recovery (PITR), restore to a NEW instance (or clone) at a timestamp before a deliberate `DELETE`, verify row counts against the primary, then delete the restored instance.
- **Friday:** **Use Case & Architecture:** Architect a zero-downtime migration strategy for a monolithic on-premise ERP system to Cloud SQL.

Resources: Cloud SQL docs (https://cloud.google.com/sql/docs); Data Engineer Handbook (https://github.com/DataExpert-io/data-engineer-handbook). **Self-check:**

- What is the failover mechanism behind Cloud SQL high availability?
- Why must PITR be restored to a new instance rather than in place?
- When is a private IP connection required for Cloud SQL?
- What does AlloyDB's columnar engine accelerate compared to stock PostgreSQL?
- How does DMS achieve minimal downtime during migration?

Week 3: Global Relational Scaling (Cloud Spanner)

Outcomes: design hotspot-free Spanner primary keys, model interleaved parent-child schemas, interpret query execution plans, explain TrueTime consistency guarantees

- **Monday:** Spanner concepts: Nodes, Replicas, TrueTime, and Strong Global Consistency.
- **Tuesday:** Schema Design: Primary keys, Interleaved Tables, and Indexing.
- **Wednesday:** Spanner performance tuning and query execution plans.
- **Thursday:** **Hands-on Project:** Create a multi-region Spanner instance, design an interleaved schema for an e-commerce cart, and load 10,000 synthetic records.
- **Friday:** **Use Case & Architecture:** Design a globally distributed payment processing ledger that requires 99.999% availability and zero data loss across continents.

Resources: Cloud Spanner docs (https://cloud.google.com/spanner/docs); Data Engineer Handbook (https://github.com/DataExpert-io/data-engineer-handbook). **Self-check:**

- What does TrueTime guarantee that wall-clock timestamps cannot?
- Why should primary keys avoid monotonically increasing values in Spanner?
- What problem do interleaved tables solve?
- How does Spanner deliver strong consistency across regions?
- When is Spanner the wrong choice versus Cloud SQL?

Week 4: Wide-Column NoSQL (Cloud Bigtable)

Outcomes: design salted row keys that avoid hotspotting, route traffic with App Profiles, diagnose hotspots in Key Visualizer, tune garbage-collection policies

- **Monday:** Bigtable architecture: Tablets, Nodes, and App Profiles.
- **Tuesday:** Row Key Design: Salting, hashing, and avoiding hotspotting.
- **Wednesday:** Performance optimization, garbage collection, and Key Visualizer.
- **Thursday:** **Hands-on Project:** Deploy a Bigtable cluster, write a Python script to ingest IoT temperature sensor data using a reversed-timestamp row key.
- **Friday:** **Use Case & Architecture:** Design an ingestion and serving layer for a financial trading platform processing 100,000 stock ticks per second.

Resources: Cloud Bigtable docs (https://cloud.google.com/bigtable/docs); Data Engineering Zoomcamp (https://github.com/DataTalksClub/data-engineering-zoomcamp). **Self-check:**

- Why do sequential row keys cause hotspotting in Bigtable?
- What does salting a row key accomplish, and what does it cost?

- What is an App Profile used for?
- How does Key Visualizer help diagnose performance problems?
- What do garbage-collection policies control in a column family?

CAPSTONE PROJECT - Milestone 1: Build an E-Commerce Multi-Tier Data Backend. Deploy Cloud SQL for user accounts, Spanner for global inventory/checkout transactions, Bigtable for clickstream tracking, and Cloud Storage for product images. Map the VPC peering required to connect them securely.

Acceptance Criteria:
- All four stores deployable from a clean environment via scripts/README in the git repo.
- Architecture diagram committed showing the VPC topology and private connectivity.
- At least 3 automated validation tests (e.g., failover check, Spanner consistency read, Bigtable hotspot probe) with a failing-case demonstration.
- Monitoring/alerting evidence: exported Cloud Monitoring dashboards or alert policies for each store.
- Monthly cost estimate with 2 optimization decisions documented (e.g., storage classes, Spanner node sizing).
- Security review: least-privilege service accounts, no hardcoded secrets, private IPs only, PII handling noted.

Milestone 1 Capstone: Reference Solution Sketch

- **Architecture:** Cloud SQL (users) + Spanner (inventory/checkout, multi-region) + Bigtable (clickstream) + GCS (product images), all on private IPs inside one VPC — each store matched to its workload profile, nothing over-provisioned.
- **Steps:** (1) provision all four stores via Terraform/gcloud scripts; (2) load the Olist dataset with seeded generators; (3) configure private IP and authorized networks; (4) run the failover, Spanner consistency, and Bigtable hotspot tests; (5) export Cloud Monitoring dashboards; (6) write the cost estimate.
- **Hardest part — a hotspot-free Bigtable row key:**

```
import hashlib
def click_row_key(user_id: str, event_ts: int) -> str:
    # salt spreads writes across tablets; reversed ts sorts newest first
    salt = hashlib.md5(user_id.encode()).hexdigest()[:4]
    return f"{salt}#{user_id}#{9999999999 - event_ts}"
```

- **Pitfall:** monotonically increasing order IDs as Spanner primary keys funnel writes onto one split — use UUIDs or bit-reversed sequences.
- **Pitfall:** leaving Cloud SQL on its default public IP fails the private-connectivity security review.
- **Pitfall:** reading Bigtable without an App Profile makes single-cluster failover tests meaningless.

2 Milestone 2: Data Warehousing & Analytics (Month 2)

Objective: Deep dive into BigQuery optimization, Lakehouses, and Data Sharing.

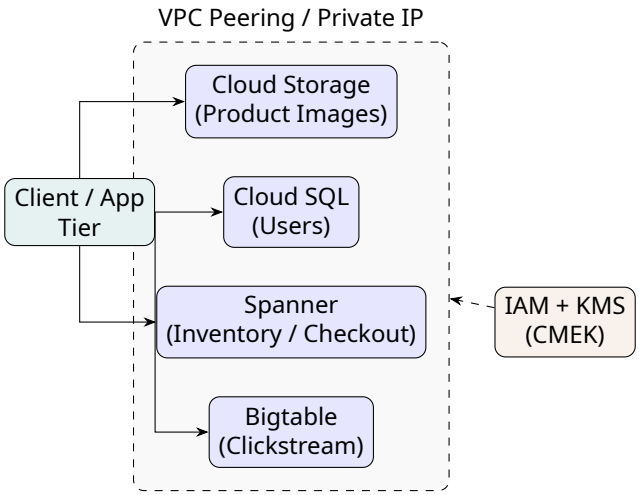

Figure 1: Milestone 1 reference architecture: multi-tier transactional and storage backend inside a VPC.

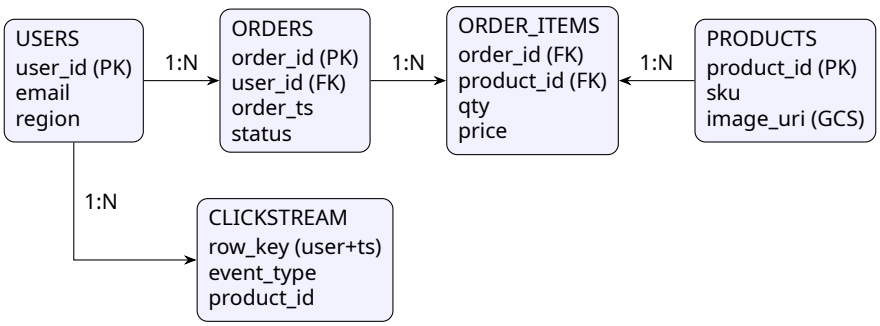

Figure 2: Capstone data model: e-commerce multi-tier backend (Cloud SQL users, Spanner orders, Bigtable clickstream).

Learning outcomes: optimize BigQuery with partitioning and clustering, estimate and control query costs, build BigLake lakehouse tables with fine-grained security, publish governed datasets via Analytics Hub

Week 5: BigQuery Architecture & Core SQL

Outcomes: apply window functions, arrays, and structs in BigQuery SQL, model a star schema with an SCD2 dimension, choose between on-demand and Editions pricing

- **Monday:** BigQuery decoupled architecture: Colossus, Jupiter, Dremel.
- **Tuesday:** Pricing models (On-demand vs. Capacity/Editions) and Slots.
- **Wednesday:** Advanced SQL (window functions, arrays, structs) and Kimball dimensional modeling: declaring the grain, fact types (transaction, periodic snapshot, accumulating), conformed dimensions, and SCD Type 1 vs. Type 2.

```
-- SCD Type 2 upsert into a customer dimension on BigQuery
MERGE analytics.dim_customer T
USING staging.customers_daily S
ON T.customer_id = S.customer_id AND T.is_current = TRUE
WHEN MATCHED AND (T.email <> S.email OR T.segment <> S.segment) THEN
  UPDATE SET valid_to = CURRENT_DATE(), is_current = FALSE
WHEN NOT MATCHED THEN
  INSERT (customer_id, email, segment, valid_from, valid_to, is_current)
  VALUES (S.customer_id, S.email, S.segment, CURRENT_DATE(), DATE '9999-12-31', TRUE);
-- Follow with a second INSERT for the new versions of the matched rows.
```

- **Thursday:** **Hands-on Project:** Query the GitHub public dataset, unnest arrays, and calculate a 7-day rolling average of commits per repository using window functions.
- **Friday:** **Use Case & Architecture:** Estimate the monthly cost of a marketing analytics workload scanning 50TB daily and recommend an Edition tier.

Option	Billing model	Best for
On-demand	Per-TiB scanned	Spiky, unpredicta low-volume workl
Editions (slots)	Slot-hours, per edition tier (Standard / Enterprise / Enterprise Plus)	Steady pro workloads n predictable spen autoscaling
Flat-rate (legacy)	Fixed monthly slots	Legacy contracts ing to Editions
BigQuery Omni	Per-TiB (or slots) on cross-cloud data in AWS/Azure	Analytics over da must stay in a cloud

Resources: BigQuery docs (https://cloud.google.com/bigquery/docs); Data Engineering Zoomcamp (https://github.com/DataTalksClub/data-engineering-zoomcamp). **Self-check:**

- Which BigQuery components make storage and compute independently scalable?
- When do Editions beat on-demand pricing for a steady workload?
- What does "declaring the grain" of a fact table commit you to?
- When is SCD Type 2 required over Type 1, and what columns implement it?
- Why must a MERGE for SCD2 match on both key and is_current?

Week 6: BigQuery Optimization

Outcomes: partition and cluster large tables, measure bytes-billed deltas across scan strategies, read execution plans for skew, decide when materialized views or BI Engine pay off

- **Monday:** Partitioning: Ingestion-time vs. Column-based (Time/Integer).
- **Tuesday:** Clustering: Sorting data for optimal filter performance.
- **Wednesday:** Materialized Views, BI Engine, and Query execution plan analysis.
- **Thursday:** **Hands-on Project:** Create a massive synthetic table. Compare the execution time and bytes billed between a full scan, a partitioned scan, and a clustered scan.
- **Friday:** **Use Case & Architecture:** Re-architect a poorly performing multi-petabyte log analysis table that is currently causing high query costs for the security team.

> **Production Note:** *Monitor BigQuery slot contention with Cloud Monitoring metrics (e.g., slots/allocated vs. slots/total_available per reservation) and alert on sustained saturation; move steady-state workloads to capacity editions with baseline plus autoscaling slots to avoid on-demand bill shock.*

Resources: BigQuery performance best practices (https://cloud.google.com/bigquery/docs/best-practices-performance-overview); DuckDB repo (https://github.com/duckdb/duckdb).
Self-check:
- How do partitioning and clustering reduce bytes billed differently?
- When does clustering beat partitioning for selective filters?
- What does a materialized view precompute, and what does it cost?
- Where in the execution plan do you spot a skewed stage?
- What does BI Engine accelerate, and when is it wasted?

Week 7: Advanced Data Types & Federated Queries

Outcomes: create external tables over GCS, run GEOGRAPHY spatial joins, evaluate BigQuery Omni egress and latency trade-offs, decide when federation beats ingestion

- **Monday:** BigQuery GIS (Geospatial) and BigQuery Search.
- **Tuesday:** External Tables and querying data directly in Cloud Storage.
- **Wednesday:** BigQuery Omni (Cross-cloud analytics on AWS/Azure).
- **Thursday:** **Hands-on Project:** Create an external table pointing to CSVs in GCS. Write a query joining this external data with native BigQuery GIS taxi-dropoff data.
- **Friday:** **Use Case & Architecture:** Design a multi-cloud data architecture for a retail company that has historical sales in AWS S3 but wants to analyze them in GCP.

Resources: BigQuery GIS docs (https://cloud.google.com/bigquery/docs/geospatial-intro); Data Engineer Handbook (https://github.com/DataExpert-io/data-engineer-handbook).
Self-check:

- What is the difference between an external table and a native BigQuery table?
- Which GEOGRAPHY function tests whether a point falls inside a polygon?
- What extra cost dimension does BigQuery Omni introduce?
- Why can external tables not benefit from clustering?
- When is federated querying preferable to ingestion?

Week 8: Data Lakehouse & Analytics Hub

Outcomes: build BigLake tables with row- and column-level security, index unstructured files with Object Tables, publish governed datasets through Analytics Hub, design a data clean room

- **Monday:** BigLake: Unifying data lakes and data warehouses with fine-grained security.
- **Tuesday:** Object Tables: Analyzing unstructured data (PDFs/Images).
- **Wednesday:** Analytics Hub: Secure data sharing and Data Clean Rooms.
- **Thursday:** **Hands-on Project:** Set up a BigLake table with column-level access control. Publish a dataset to the Analytics Hub for a "partner" organization.
- **Friday:** **Use Case & Architecture:** Architect a Data Clean Room for an ad agency to merge their CRM data with Google Ads data without exposing raw PII.

Resources: BigLake docs (https://cloud.google.com/bigquery/docs/biglake-intro); Apache Iceberg repo (https://github.com/apache/iceberg). **Self-check:**

- How does BigLake give GCS files warehouse-grade security?
- What can an Object Table index that a regular external table cannot?
- How does Analytics Hub share data without copying it?
- What does a data clean room prevent each party from seeing?
- Why use policy tags instead of separate views per role?

CAPSTONE PROJECT - Milestone 2: Enterprise Data Lakehouse Build. Ingest 5 years of retail data into GCS. Create BigLake tables over the data, apply row/column level security policies based on region, create Materialized Views for executive dashboards, and set up a BI Engine reservation.

Acceptance Criteria:

- End-to-end lakehouse rebuilds from a clean environment following the repo README (ingest to GCS, BigLake, MVs).
- Architecture diagram committed (draw.io/TikZ/PNG) showing zones, security policies, and serving layers.
- At least 3 automated data-quality tests on the curated tables (row counts, null checks, freshness) with a failing-case demonstration.
- Monitoring/alerting evidence: query-cost and slot-utilization dashboards or exported alert policies.
- Monthly cost estimate (on-demand vs. editions) with 2 optimization decisions documented (partitioning, clustering, MV, or BI Engine).
- Security review: row/column-level policies verified with a negative test, least-privilege roles, no hardcoded secrets.

Milestone 2 Capstone: Reference Solution Sketch

- **Architecture:** GCS Parquet zones → BigLake tables with policy tags and row access policies → BigQuery materialized views + BI Engine → Analytics Hub listing — one governed copy serving both internal dashboards and external partners.
- **Steps:** (1) land 5 years of retail Parquet in GCS partitioned by date; (2) create BigLake tables over the files; (3) apply a row access policy on `region` and a policy tag on customer PII columns; (4) build MVs for executive KPIs and add a BI Engine reservation; (5) publish the curated dataset via Analytics Hub; (6) run the negative security test.
- **Hardest part — row-level security that survives audits:**

```
CREATE ROW ACCESS POLICY region_filter
ON lakehouse.fact_sales
GRANT TO ("group:analysts-eu@example.com")
FILTER USING (region = "EU");
-- negative test: query as an analyst-us member must return zero EU rows
```

- **Pitfall:** clustering on a low-cardinality column yields no pruning — cluster on the columns selective filters actually use.
- **Pitfall:** materialized views support a restricted SQL subset (no arbitrary joins or UDFs) — keep them simple pre-aggregations.
- **Pitfall:** policy tags deny by default; a forgotten group grant blocks dashboards silently — always pair with a negative test.

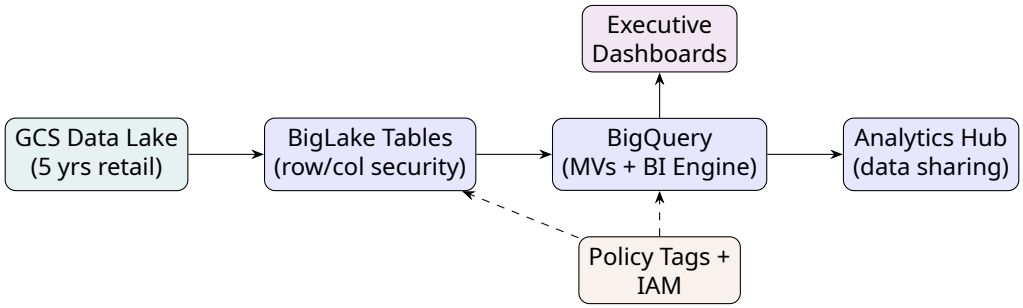

Figure 3: Milestone 2 reference architecture: governed lakehouse from GCS to shared analytics.

3 Milestone 3: Batch Processing & ETL Orchestration (Month 3)

Objective: Master Hadoop migration, No-code ETL, CDC, and Airflow.
Learning outcomes: migrate Hadoop workloads to Dataproc, build no-code Data Fusion pipelines, configure Datastream CDC replication, orchestrate multi-service DAGs in Cloud Composer

Week 9: Managed Hadoop/Spark (Cloud Dataproc)

Outcomes: submit PySpark jobs to ephemeral clusters, migrate HDFS data to GCS, tune Spark memory and autoscaling, make Spark writes idempotent under preemption

- **Monday:** Dataproc Architecture, Ephemeral clusters, and Preemptible VMs.

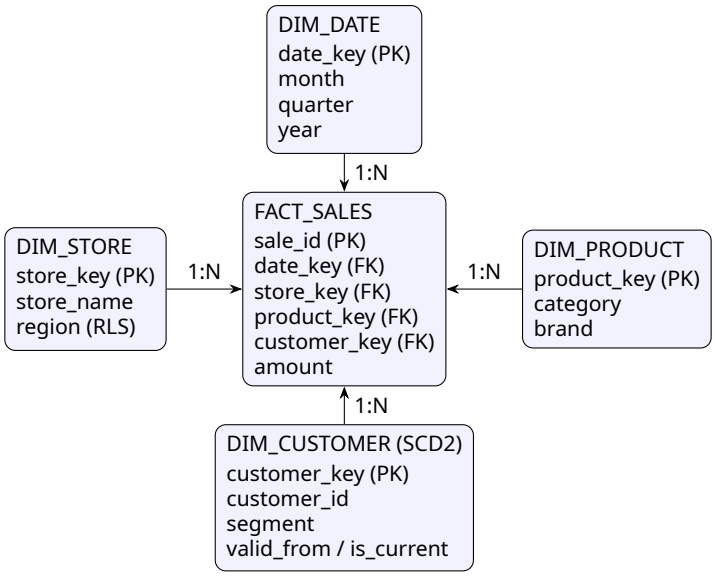

Figure 4: Capstone data model: retail lakehouse star schema with conformed dimensions and SCD2 customers.

- **Tuesday:** Migrating from on-prem HDFS to Cloud Storage.
- **Wednesday:** Serverless Dataproc and Spark job tuning.
- **Thursday:** **Hands-on Project:** Deploy an ephemeral Dataproc cluster, submit a PySpark job that reads from GCS, aggregates sales data, writes back to GCS, and self-deletes.
- **Friday:** **Use Case & Architecture:** Design a migration strategy for a 100-node on-premise Cloudera cluster, prioritizing cost reduction and decoupling compute from storage.

Production Note: *Make Spark jobs idempotent (deterministic output paths, write-then-swap into BigQuery) so preemptible VM reclamation or retries never double-load data; use ephemeral clusters and autoscaling policies as a cost guardrail.*

Resources: Cloud Dataproc docs (`https://cloud.google.com/dataproc/docs`); Apache Spark repo (`https://github.com/apache/spark`). **Self-check:**

- Why should Dataproc clusters be ephemeral for scheduled jobs?
- What is the cost trade-off of preemptible VMs in a Spark cluster?
- Why migrate HDFS data to GCS instead of keeping HDFS?
- What Spark property most often explains executor out-of-memory errors?
- How do autoscaling policies decide when to add workers?

Week 10: Visual ETL (Cloud Data Fusion)

Outcomes: build Wrangler-based no-code pipelines, parameterize deployed pipelines with macros, schedule recurring runs, justify Data Fusion versus Dataflow for analyst teams

- **Monday:** CDAP concepts, Data Fusion architecture, and Private IPs.
- **Tuesday:** Wrangler: Interactive data cleaning and transformation.
- **Wednesday:** Plugins, Macros, and Scheduling pipelines.

- **Thursday: Hands-on Project:** Build a no-code Data Fusion pipeline that reads messy customer data from GCS, uses Wrangler to extract domains from emails, and writes to BigQuery.
- **Friday: Use Case & Architecture:** Evaluate Data Fusion vs. Dataflow for a team of business analysts who need to build daily reporting pipelines without writing Java/Python.

Resources: Cloud Data Fusion docs (https://cloud.google.com/data-fusion/docs); Data Engineering Zoomcamp (https://github.com/DataTalksClub/data-engineering-zoomcamp).
Self-check:

- What underlying framework executes Data Fusion pipelines?
- What is Wrangler's role before a pipeline runs at scale?
- When should you choose Data Fusion over Dataflow?
- Why use a private IP Data Fusion instance?
- What do macros parameterize in a deployed pipeline?

Week 11: Change Data Capture (Datastream)

Outcomes: configure Datastream CDC from PostgreSQL to BigQuery, classify schema changes as breaking or non-breaking, monitor freshness and failed events, handle source schema drift

- **Monday:** CDC concepts and the Datastream architecture.
- **Tuesday:** Source configurations (MySQL/PostgreSQL/Oracle).
- **Wednesday:** Destination configurations (Cloud Storage vs. BigQuery native) and data contracts: BigQuery schema enforcement on load (reject rows with unknown or mismatched fields), contract versioning, breaking vs. non-breaking changes (adding a nullable column vs. renaming one), and producer vs. consumer ownership of the contract.
- **Thursday: Hands-on Project:** Set up a Cloud SQL PostgreSQL database, configure logical replication, and build a Datastream pipeline to sync changes to BigQuery in near real-time. Test the contract: add a nullable column (non-breaking) and rename one (breaking), observing Datastream's schema-drift handling.
- **Friday: Use Case & Architecture:** Architect a zero-downtime database replication pipeline from an on-premise Oracle database to BigQuery for real-time BI reporting.

> **Production Note:** *Alert on Datastream CDC latency (data freshness metric) and failed events in Cloud Monitoring; write a runbook for stale log positions and backfill via GCS staging when BigQuery apply falls behind. Schema drift at the source must be handled explicitly — test adding and renaming a column.*

Resources: Datastream docs (https://cloud.google.com/datastream/docs); Data Engineer Handbook (https://github.com/DataExpert-io/data-engineer-handbook). **Self-check:**

- How does Datastream capture changes without querying the source tables?
- Which is a breaking contract change: adding a nullable column or renaming one?
- What does BigQuery schema enforcement on load reject by default?
- Who should own a data contract — producer, consumer, or both?
- What does the data freshness metric tell you about CDC lag?

Week 12: Pipeline Orchestration (Cloud Composer)

Outcomes: author Airflow DAGs with sensors and Google operators, orchestrate cross-service Dataproc and BigQuery tasks, implement idempotent retry-safe tasks, wire failure notifications

- **Monday:** Apache Airflow concepts: DAGs, Operators, Tasks, and XComs.
- **Tuesday:** Cloud Composer 2 architecture (Autoscaling, GKE underlying).
- **Wednesday:** Cross-service orchestration (Triggering Dataproc, Dataflow, BQ).
- **Thursday:** **Hands-on Project:** Write a Python DAG that: 1. Checks if a file exists in GCS. 2. Triggers a Dataproc job. 3. Loads the output to BigQuery. 4. Sends an email on failure.

```
from datetime import datetime, timedelta
from airflow import DAG
from airflow.providers.google.cloud.sensors.gcs import GCSObjectExistenceSensor
from airflow.providers.google.cloud.operators.dataproc import DataprocSubmitJobOperator
from airflow.providers.google.cloud.operators.bigquery import BigQueryInsertJobOperator
from airflow.operators.email import EmailOperator

default_args = {"retries": 2, "retry_delay": timedelta(minutes=5)}

with DAG("daily_sales_elt", schedule="@daily", start_date=datetime(2026, 1, 1),
         default_args=default_args, catchup=False) as dag:
    wait_for_file = GCSObjectExistenceSensor(
        task_id="wait_for_file", bucket="raw-sales", object="daily/{{ ds }}/sales.csv")
    spark_job = DataprocSubmitJobOperator(
        task_id="spark_transform", project_id="my-project", region="us-central1",
        job={"placement": {"cluster_name": "ephemeral-etl"},
             "pyspark_job": {"main_python_file_uri": "gs://code/transform.py"}})
    load_bq = BigQueryInsertJobOperator(
        task_id="load_bq",
        configuration={"query": {"query": "CALL analytics.refresh_daily_sales()",
                                 "useLegacySql": False}})
    notify = EmailOperator(task_id="notify", to=["data-oncall@example.com"],
                           subject="daily_sales_elt finished", html_content="Done.",
                           trigger_rule="all_done")
    wait_for_file >> spark_job >> load_bq >> notify
```

- **Friday:** **Use Case & Architecture:** Design a resilient, self-healing orchestration layer for a daily banking reconciliation process spanning 15 dependent systems.

Production Note: *Enable Composer DAG failure notifications: set email/Slack on_failure_callbacks and Cloud Monitoring alert policies on failed task instances and SLA misses; use retries with exponential backoff and keep tasks idempotent so re-runs are safe.*

Resources: Apache Airflow repo (`https://github.com/apache/airflow`); Cloud Composer docs (`https://cloud.google.com/composer/docs`). **Self-check:**

- Why must Airflow tasks be idempotent for safe retries?
- What does the `all_done` trigger rule guarantee here?
- What runs underneath Cloud Composer 2?
- When should you use a GCS sensor instead of a schedule?
- What information should flow through XComs — and what should not?

CAPSTONE PROJECT - Milestone 3: Automated Batch ELT Pipeline. Use Cloud Scheduler to trigger a Composer DAG. The DAG runs a Datastream CDC sync verification, triggers a Serverless Dataproc Spark job to calculate complex ML features, loads them into BigQuery, and creates a Materialized View.

Acceptance Criteria:

- End-to-end pipeline runs from a clean environment following the repo README (Scheduler triggers Composer; DAG completes without manual steps).
- Architecture diagram committed showing Scheduler, Composer, Datastream, Dataproc, and BigQuery flows.
- At least 3 automated data-quality tests after load (CDC row-count reconciliation, freshness check, feature null-rate) with a failing-case demonstration.
- Monitoring/alerting evidence: DAG failure notification and a Cloud Monitoring alert on pipeline latency.
- Monthly cost estimate with 2 optimization decisions documented (ephemeral clusters, preemptible VMs, Composer sizing).
- Security review: service accounts scoped per service, no hardcoded secrets, PII handling noted.

Milestone 3 Capstone: Reference Solution Sketch

- **Architecture:** Cloud Scheduler → Composer DAG → (Datastream freshness check → serverless Dataproc Spark feature job → BigQuery load → MV refresh) — one orchestrator, each stage idempotent.
- **Steps:** (1) verify CDC row counts between source and BigQuery; (2) run the Spark aggregation on an ephemeral cluster; (3) load with `WRITE_TRUNCATE` on a date partition; (4) refresh the materialized view; (5) wire failure email and latency alerts; (6) document the backfill runbook.
- **Hardest part — an idempotent Spark write safe under retries:**

```
ds = kwargs["ds"]  # logical date from Airflow context
out = f"gs://ml-features/dt={ds}"  # deterministic path per run
(spark.read.parquet(f"gs://cdc-staging/dt={ds}")
      .groupBy("customer_id")
      .agg(F.count("*").alias("orders_30d"),
           F.sum("amount").alias("spend_30d"))
      .write.mode("overwrite").parquet(out))
# BQ load uses WRITE_TRUNCATE on partition dt=ds: re-runs replace, never duplicate
```

- **Pitfall:** `catchup=True` on a new DAG floods weeks of backfill runs at once — disable it or cap `max_active_runs`.
- **Pitfall:** append-mode loads double-count rows when a task retries — always overwrite by partition.
- **Pitfall:** passing DataFrames or file contents through XComs stalls the scheduler — pass paths and row counts only.

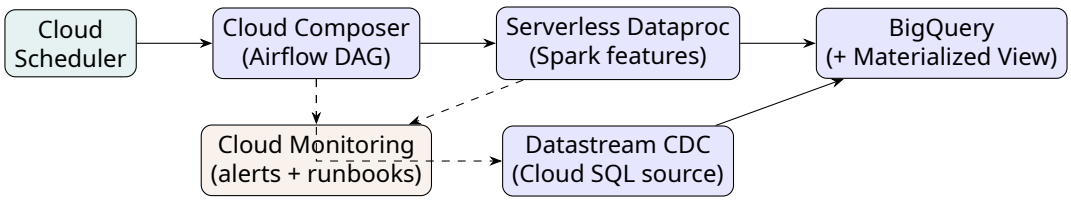

Figure 5: Milestone 3 reference architecture: scheduled batch ELT with CDC verification.

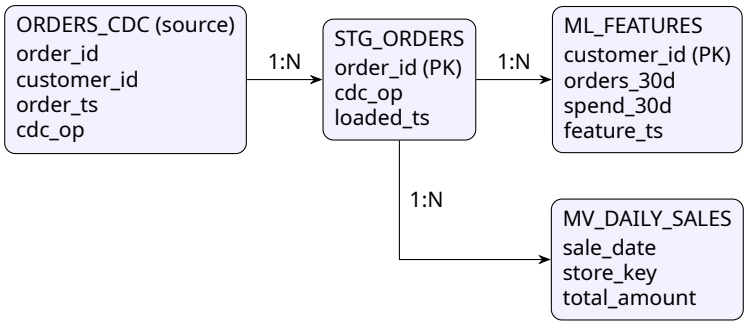

Figure 6: Capstone data model: batch ELT flow from CDC staging to ML features and materialized aggregates.

4 Milestone 4: Streaming & Real-Time Analytics (Month 4)

Objective: Conquer unbounded data, message queues, and real-time visualization.
Learning outcomes: design Pub/Sub topics with dead-letter handling, implement Beam streaming pipelines with windowing and late-data handling, unit-test and monitor Dataflow jobs in production, serve real-time dashboards in Looker

Week 13: Asynchronous Messaging (Pub/Sub)

Outcomes: publish with retry and exponential backoff, configure dead-letter topics and snapshot/seek replay, enforce topic schemas at the edge, deduplicate at-least-once delivery downstream

- **Monday:** Pub/Sub concepts: Topics, Subscriptions (Push/Pull), Acknowledgements; schema validation with Avro or Protocol Buffer schemas attached to topics (rejecting non-conforming publishes as contract enforcement at the edge).
- **Tuesday:** Dead-Letter Topics, Retry Policies, and Snapshot/Seek.
- **Wednesday:** Pub/Sub Lite vs. Pub/Sub Standard (Cost vs. Operational overhead).
- **Thursday:** **Hands-on Project:** Create a Pub/Sub topic and pull subscription. Write a Python publisher to stream mock financial transactions. Configure a Dead-Letter Topic for failed messages.

```
import json, random, time
from google.cloud import pubsub_v1

publisher = pubsub_v1.PublisherClient(
    publisher_options=pubsub_v1.types.PublisherOptions(
        retry=pubsub_v1.types.Retry(initial=0.1, maximum=10.0, multiplier=2.0)))
topic_path = publisher.topic_path("my-project", "card-transactions")

def publish(txn):
    future = publisher.publish(topic_path, json.dumps(txn).encode("utf-8"),
                               origin="pos-simulator")
    future.add_done_callback(lambda f: print("acked:", f.result()))

for i in range(1000):
    publish({"card_id": f"card-{random.randint(1, 500)}",
             "amount": round(random.uniform(1, 500), 2),
             "city": random.choice(["NYC", "LON", "TYO"]), "ts": time.time()})

# DLQ is configured on the subscription, not the publisher:
```

```
# gcloud pubsub subscriptions create card-sub --topic=card-transactions \
#   --dead-letter-topic=card-dlq --max-delivery-attempts=5
```

- **Friday:** **Use Case & Architecture:** Architect a global telemetry ingestion point for 5 million mobile games, ensuring exactly-once processing semantics.

Production Note: *Publishing is at-least-once: deduplicate downstream with message IDs or Dataflow exactly-once mode, publish with retry and exponential backoff, and alert on oldest unacked message age and DLQ message counts.*

Resources: Pub/Sub docs (https://cloud.google.com/pubsub/docs); Apache Kafka repo (https://github.com/apache/kafka). **Self-check:**

- What happens to a message that exceeds max delivery attempts?
- How does a topic schema enforce a data contract at publish time?
- Why is Pub/Sub publishing at-least-once, and how do you deduplicate?
- When does Pub/Sub Lite beat Standard on cost?
- What does the oldest-unacked-message-age metric warn you about?

Week 14: Dataflow & Apache Beam (Basics)

Outcomes: implement PTransforms and ParDos, unit-test pipelines with TestPipeline and PAssert, package Flex Templates, run and autoscale a job on the Dataflow runner

- **Monday:** Apache Beam model: PCollections, PTransforms, ParDo, and Runners.
- **Tuesday:** Dataflow architecture: Streaming Engine, Shuffle, Autoscaling.
- **Wednesday:** Dataflow Templates (Classic vs. Flex Templates).
- **Thursday:** **Hands-on Project:** Write an Apache Beam pipeline in Python that reads texts, tokenizes words, counts frequencies (WordCount), and runs it on the Dataflow runner.
- **Friday:** **Use Case & Architecture:** Design a pipeline to anonymize incoming healthcare HL7 messages in real-time before storing them in BigQuery.

Production Note: *Unit-test Beam pipelines before deploying to Dataflow: run transforms locally with TestPipeline and assert outputs with PAssert (beam.testing); test DoFns in isolation and use TestStream to verify windowed and late-data behavior in CI.*

Resources: Apache Beam docs (https://beam.apache.org/documentation/); Data Engineering Zoomcamp (https://github.com/DataTalksClub/data-engineering-zoomcamp). **Self-check:**

- What is the difference between a PTransform and a ParDo?
- Why unit-test Beam with TestPipeline before deploying to Dataflow?
- What does Streaming Engine move off the worker VMs?
- When should you package a pipeline as a Flex Template?
- How does Dataflow autoscaling react to a growing backlog?

Week 15: Dataflow Advanced (Windowing, State & Exactly-Once)

Outcomes: choose fixed, sliding, or session windows per workload, configure triggers and allowed lateness, manage per-key state with the Beam State & Timers API, join streams over windows with CoGroupByKey, reason about end-to-end exactly-once versus at-least-once plus dedup, alert on watermark lag

- **Monday:** Event Time vs. Processing Time and Watermarks.
- **Tuesday:** Windowing strategies: Fixed, Sliding, and Session windows.
- **Wednesday:** Handling Late Data: Allowed Lateness and Triggers. Stateful streaming: Beam State & Timers API, per-key state bags, and state TTL via watermark garbage collection — why unbounded state kills streaming jobs.

```
import apache_beam as beam
from apache_beam.coders import coders
from apache_beam.transforms.userstate import BagStateSpec, TimerSpec, on_timer

class DedupWithTimer(beam.DoFn):
    SEEN = BagStateSpec("seen", coders.StrUtf8Coder())
    EXPIRY = TimerSpec("expiry")

    def process(self, element, seen=beam.DoFn.StateParam(SEEN),
                expiry=beam.DoFn.TimerParam(EXPIRY)):
        key, event = element
        expiry.set(event["event_ts"] + 600)  # state TTL: 10 min
        if event["event_id"] in set(seen.read()):
            return  # duplicate: discard
        seen.add(event["event_id"])
        yield event

    @on_timer(EXPIRY)
    def on_expiry(self, seen=beam.DoFn.StateParam(SEEN)):
        seen.clear()  # free per-key state at timer fire
```

- **Thursday:** **Hands-on Project:** Build a streaming Dataflow pipeline reading from Pub/Sub. Group user clicks into 5-minute Session Windows, allowing for 2 minutes of late data.

```
import json
import apache_beam as beam
from apache_beam.options.pipeline_options import PipelineOptions, StandardOptions
from apache_beam.transforms.window import TimestampedValue, FixedWindows
import apache_beam.transforms.trigger as trigger

options = PipelineOptions(project="my-project", region="us-central1",
                          temp_location="gs://my-bucket/tmp")
options.view_as(StandardOptions).streaming = True

with beam.Pipeline(options=options) as p:
    (p | "Read" >> beam.io.ReadFromPubSub(
             subscription="projects/my-project/subscriptions/clicks-sub")
       | "Parse" >> beam.Map(lambda b: json.loads(b.decode("utf-8")))
       | "Stamp" >> beam.Map(lambda e: TimestampedValue(e, e["event_ts"]))
       | "Window" >> beam.WindowInto(
             FixedWindows(300),  # 5-minute windows
             trigger=trigger.AfterWatermark(late=trigger.AfterCount(1)),
             allowed_lateness=120,  # 2 minutes of late data
             accumulation_mode=trigger.AccumulationMode.DISCARDING)
       | "KeyByUser" >> beam.Map(lambda e: (e["user_id"], 1))
       | "Count" >> beam.CombinePerKey(sum)
       | "Write" >> beam.io.WriteToBigQuery(
             "my-project:analytics.clicks_5min",
             create_disposition=beam.io.BigQueryDisposition.CREATE_IF_NEEDED))
```

- **Friday:** Stream-stream joins: CoGroupByKey and joins over windows, join + watermark alignment, and state retention implications. End-to-end exactly-once: Dataflow exactly-once shuffle with Streaming Engine and idempotent sinks (BigQuery Storage Write API

exactly-once semantics) versus at-least-once plus downstream dedup. **Use Case & Architecture:** Troubleshoot a scenario where a streaming dashboard is showing inaccurate daily aggregations due to mobile devices going offline and uploading data late.

Production Note: *Create Cloud Monitoring alert policies on Dataflow system lag and per-stage data watermark lag; growing watermark lag signals late data or a stuck stage — page on-call and check the upstream Pub/Sub backlog (oldest unacked message age).*

Production Note: *Changing stateful logic (state specs, timers, or windowing) usually breaks in-place job updates because existing state cannot map to the new schema: prefer an in-place update only for stateless-compatible changes, and use drain-and-replace (drain the old job, start the new one from a snapshot or fresh subscription) when stateful transforms change — plan the cutover to avoid reprocessing or gaps.*

Resources: Beam programming guide (https://beam.apache.org/documentation/programming-guide/); OpenLineage repo (https://github.com/OpenLineage/OpenLineage). **Self-check:**

- What does a watermark estimate, and what lies behind it?
- Why does unbounded per-key state kill a streaming job, and how do timers plus watermark garbage collection bound it?
- In the dedup DoFn, what do the `SEEN` bag state and `EXPIRY` timer each do?
- What must align across both inputs for a CoGroupByKey stream-stream join to emit, and what state does it retain?
- How do Streaming Engine shuffle and the BigQuery Storage Write API deliver end-to-end exactly-once, and when is at-least-once plus downstream dedup the cheaper choice?

Week 16: Business Intelligence & Looker

Outcomes: model governed metrics in LookML explores, build drill-down dashboards on streaming data, push reverse-ETL segments to operational APIs, compare Looker Studio with Looker Core

- **Monday:** Looker Architecture: The Semantic Layer and LookML.
- **Tuesday:** Dimensions, Measures, Explores, and Views.
- **Wednesday:** Looker Studio vs. Looker Core (Self-serve vs. Governed BI); Reverse ETL: pushing curated BigQuery segments back into operational CRM/marketing APIs via a scheduled query plus Cloud Function (or a Dataform operation), with identity resolution across systems and sync-frequency trade-offs (freshness vs. API rate limits and cost).
- **Thursday:** **Hands-on Project:** Connect Looker Studio (or a Looker trial) to your BigQuery retail dataset. Create a real-time dashboard monitoring streaming sales with drill-down filters. Reverse-ETL lab: schedule a BigQuery query that materializes a high-value customer segment and have a Cloud Function POST it to a mock marketing API webhook.
- **Friday:** **Use Case & Architecture:** Design the BI access layer for an enterprise where 500 analysts need to query centralized data without creating conflicting metrics (e.g., "Gross Revenue" definitions).

Resources: Looker docs (https://cloud.google.com/looker/docs); DataHub repo (https://github.com/datahub-project/datahub). **Self-check:**

- Why does a LookML semantic layer prevent conflicting metric definitions?
- What problem does reverse ETL solve that dashboards cannot?

- What breaks in a sync without identity resolution across systems?
- What trade-off governs reverse-ETL sync frequency?
- When is Looker Core justified over Looker Studio?

CAPSTONE PROJECT - Milestone 4: Real-Time Fraud Detection. Build a Python script simulating credit card swipes sent to Pub/Sub. Create a Dataflow pipeline that uses a Sliding Window to detect if a card is used in two different cities within 10 minutes, flags it, and streams the alert to BigQuery for live Looker visualization.

Acceptance Criteria:
- End-to-end streaming pipeline runs from a clean environment following the repo README (simulator to Pub/Sub to Dataflow to BigQuery/Looker).
- Architecture diagram committed, including DLQ, windowing semantics, and serving layer.
- At least 3 automated tests (PAssert unit tests for the fraud-detection logic plus a late-data case) with a failing-case demonstration.
- Monitoring/alerting evidence: alert policies on Dataflow watermark lag and Pub/Sub backlog (screenshot or exported config).
- Monthly cost estimate with 2 optimization decisions documented (Streaming Engine, autoscaling bounds, Pub/Sub vs. Lite).
- Security review: least-privilege runner service account, no hardcoded secrets, PII handling noted (card data pseudonymized).

Milestone 4 Capstone: Reference Solution Sketch

- **Architecture:** Python card-swipe simulator → Pub/Sub (with DLQ) → Dataflow sliding-window fraud rule → BigQuery alerts table → Looker live dashboard — serverless end to end, no clusters to manage.
- **Steps:** (1) generate seeded transactions with occasional cross-city anomalies; (2) publish with retry/backoff; (3) window events per card with a 10-minute sliding window; (4) flag windows containing two distinct cities; (5) write alerts to BigQuery; (6) build the Looker dashboard and watermark-lag alert.
- **Hardest part — the two-cities-in-ten-minutes rule:**

```
alerts = (
    events
    | "KeyByCard" >> beam.Map(lambda e: (e["card_id"], e))
    | "Window" >> beam.WindowInto(SlidingWindows(600, 60),
          allowed_lateness=120,
          accumulation_mode=trigger.AccumulationMode.DISCARDING)
    | "Group" >> beam.GroupByKey()
    | "Flag" >> beam.FlatMap(lambda kv: [(kv[0], w)]
        for w in [1] if len({e["city"] for e in kv[1]}) > 1))
```

- **Pitfall:** keying on processing time instead of event time misses offline-mobile swipes — always use the event timestamp.
- **Pitfall:** default watermarks silently drop late events; set `allowed_lateness` and monitor watermark lag.
- **Pitfall:** unbounded per-key state in session windows grows worker memory — emit and clear state per window firing.

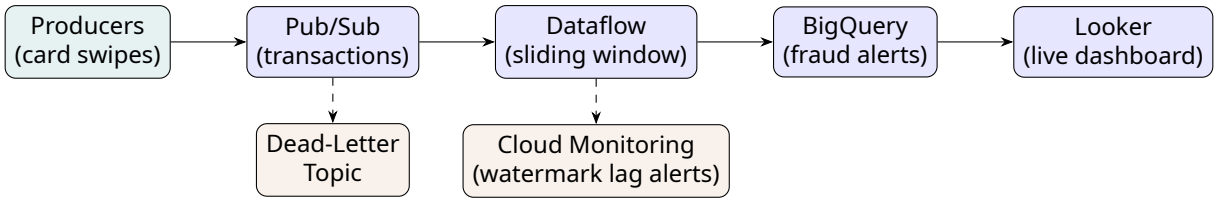

Figure 7: Milestone 4 reference architecture: real-time fraud detection streaming pipeline.

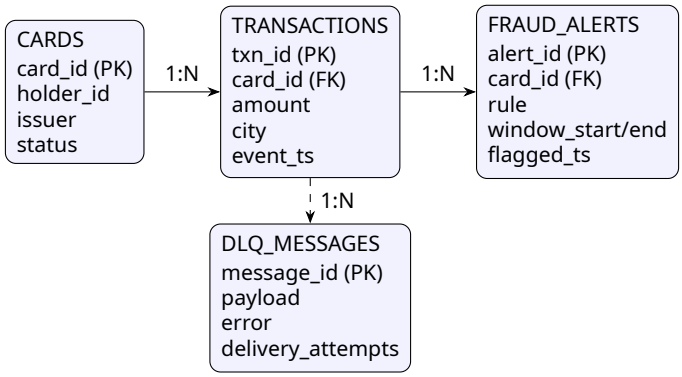

Figure 8: Capstone data model: real-time fraud detection (cards, transactions, alerts, dead-lettered messages).

5 Milestone 5: Machine Learning Operationalization (Month 5)

Objective: Embed AI into data pipelines and manage the MLOps lifecycle.
Learning outcomes: train and evaluate models in BigQuery ML, deploy AutoML models to Vertex AI endpoints, orchestrate Kubeflow pipelines with continuous training triggers, integrate AI APIs into streaming pipelines

Week 17: BigQuery ML (BQML)

Outcomes: train classification and clustering models in pure SQL, forecast with ARIMA_PLUS, evaluate with ML.EVALUATE ROC and precision/recall, detect overfitting from eval output

- **Monday:** BQML capabilities: Regression, Classification, K-Means Clustering.
- **Tuesday:** Time-series forecasting (ARIMA_PLUS) and Anomaly Detection.
- **Wednesday:** Model evaluation (ML.EVALUATE) and hyperparameter tuning.
- **Thursday:** **Hands-on Project:** Use BQML to train a logistic regression model on Google Analytics sample data to predict user churn. Evaluate the model's ROC curve using SQL.
- **Friday:** **Use Case & Architecture:** Architect a system to automatically forecast daily inventory needs for 1,000 retail stores using purely SQL-based pipelines.

Resources: BigQuery ML docs (https://cloud.google.com/bigquery/docs/bqml-introduction); dbt Core repo (https://github.com/dbt-labs/dbt-core). **Self-check:**

- Which BQML model type fits customer churn prediction?
- What does ARIMA_PLUS automate compared to classic ARIMA?
- Which ML.EVALUATE metrics matter most for imbalanced churn?
- What is the advantage of training where the data already lives?
- How do you detect overfitting with BQML evaluation output?

Week 18: Vertex AI Foundations

Outcomes: train an AutoML Tabular model, version it in the Model Registry, deploy to a managed endpoint, request online predictions with feature attributions

- **Monday:** Vertex AI Ecosystem, AutoML vs. Custom Training.
- **Tuesday:** Vertex AI Model Registry and Endpoint deployment (Online vs. Batch).
- **Wednesday:** Explainable AI (Feature attributions).
- **Thursday:** **Hands-on Project:** Train an AutoML Tabular model on a cleaned BigQuery dataset. Deploy the model to a Vertex AI Endpoint and send a cURL request for an online prediction.
- **Friday:** **Use Case & Architecture:** Design a workflow for data scientists to train custom TensorFlow models on GPUs while accessing data securely stored in BigQuery.

Resources: Vertex AI docs (https://cloud.google.com/vertex-ai/docs); Data Engineer Handbook (https://github.com/DataExpert-io/data-engineer-handbook). **Self-check:**

- When is AutoML Tabular sufficient versus custom training?
- What does the Model Registry version control?
- Online versus batch prediction: which serves a checkout flow?
- What do feature attributions tell a model consumer?
- Why deploy through an endpoint rather than calling the model directly?

Week 19: MLOps & Vertex AI Pipelines

Outcomes: serve features online and offline via Feature Store, compose a Kubeflow Extract-Train-Deploy pipeline, gate retraining on drift alerts, distinguish skew from drift

- **Monday:** Vertex AI Feature Store: Online vs. Offline serving.
- **Tuesday:** Kubeflow pipelines and Vertex AI Pipelines orchestration.
- **Wednesday:** Continuous Training (CT) triggers and Model Monitoring (Data Drift).
- **Thursday:** **Hands-on Project:** Build a simple Kubeflow pipeline (Extract $\rightarrow$ Train $\rightarrow$ Deploy). Compile it and run it using Vertex AI Pipelines.
- **Friday:** **Use Case & Architecture:** Architect an MLOps lifecycle that detects data drift in a production recommendation engine and automatically triggers a retraining pipeline via Cloud Functions.

> **Production Note:** *Wire Vertex Model Monitoring alerts (feature drift and prediction skew) to a notification channel and a runbook: triage the drifting feature, validate upstream schema drift, then trigger retraining only after data-quality checks pass — never auto-deploy without evaluation gates.*

Resources: Vertex AI Pipelines docs (https://cloud.google.com/vertex-ai/docs/pipelines); OpenLineage repo (https://github.com/OpenLineage/OpenLineage). **Self-check:**

- What problem does a Feature Store solve between training and serving?
- Online vs. offline serving: which backs a real-time recommender?
- What does a Kubeflow pipeline component encapsulate?
- Which signal should gate an automated retraining trigger?
- How does prediction skew differ from training-data drift?

Week 20: Unstructured Data & AI APIs

Outcomes: call Vision and Natural Language APIs from a streaming Dataflow job, handle API rate limits with backoff and batching, make OCR output searchable, forecast per-call API cost

- **Monday:** Cloud Vision API and Video Intelligence API.
- **Tuesday:** Natural Language API and Translation API.
- **Wednesday:** Integrating AI APIs within Dataflow pipelines.
- **Thursday: Hands-on Project:** Write a Dataflow pipeline that reads customer support text reviews from Pub/Sub, calls the Natural Language API to get sentiment scores, and saves to BigQuery.
- **Friday: Use Case & Architecture:** Design a scalable pipeline to process terabytes of uploaded user images, extract text (OCR), tag objects, and make the metadata searchable.

Resources: Cloud AI APIs docs (https://cloud.google.com/products/ai); Data Engineering Zoomcamp (https://github.com/DataTalksClub/data-engineering-zoomcamp). **Self-check:**

- When should a Dataflow pipeline call an AI API versus BQML?
- What rate-limit handling belongs around external API calls?
- Which API extracts entities and sentiment from support text?
- How do you make OCR output searchable in BigQuery?
- What cost risk do per-call AI APIs add to streaming jobs?

CAPSTONE PROJECT - Milestone 5: End-to-End MLOps Pipeline. Use Composer to orchestrate a pipeline that extracts features from BigQuery, pushes them to Vertex Feature Store, trains a custom model using Vertex Training, registers the model, and sets up Vertex Model Monitoring for serving skew.

Acceptance Criteria:

- End-to-end MLOps pipeline runs from a clean environment following the repo README (feature extraction to registered, monitored model).
- Architecture diagram committed covering Feature Store, training, registry, endpoint, and monitoring loop.
- At least 3 automated tests (feature-quality checks, model evaluation threshold gate, pipeline unit test) with a failing-case demonstration.
- Monitoring/alerting evidence: Vertex Model Monitoring drift/skew alert configuration exported or screenshotted.
- Monthly cost estimate with 2 optimization decisions documented (machine types, endpoint min/max nodes, batch vs. online).
- Security review: pipeline service account with least privilege, no hardcoded secrets, training-data PII handling noted.

Milestone 5 Capstone: Reference Solution Sketch

- **Architecture:** BigQuery features $\rightarrow$ Vertex Feature Store $\rightarrow$ Vertex Training $\rightarrow$ Model Registry $\rightarrow$ Endpoint, orchestrated by Composer, with Model Monitoring closing the retraining loop — one managed platform instead of bespoke infrastructure.
- **Steps:** (1) build the feature table in BigQuery; (2) sync to Feature Store; (3) train with an evaluation-metrics artifact; (4) gate registration on a quality threshold; (5) deploy to an endpoint with traffic splitting; (6) configure drift/skew alerts that trigger the retraining pipeline.

- **Hardest part — the evaluation gate that blocks bad models:**

```
@component(base_image="python:3.10")
def register_gate(evaluation: Input[Metrics]) -> bool:
    auc = evaluation.metadata["roc_auc"]
    if auc < 0.80:
        raise ValueError(f"ROC AUC {auc:.3f} below 0.80 gate")
    return True  # downstream register/deploy tasks only run if this passes
```

- **Pitfall:** auto-deploying without an evaluation gate ships regressions silently — gate on a metric, page on failure.
- **Pitfall:** features computed ad hoc in training and serving cause skew — serve both paths from Feature Store.
- **Pitfall:** `min_replica_count=0` saves money but adds cold-start latency to the first request after idle.

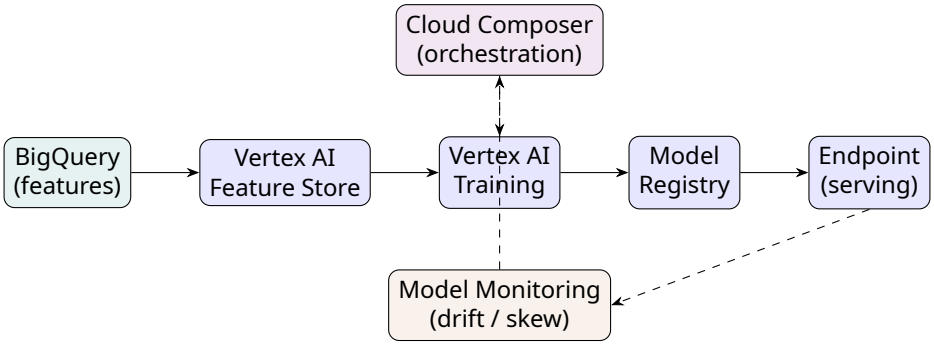

Figure 9: Milestone 5 reference architecture: MLOps lifecycle on Vertex AI with Composer orchestration.

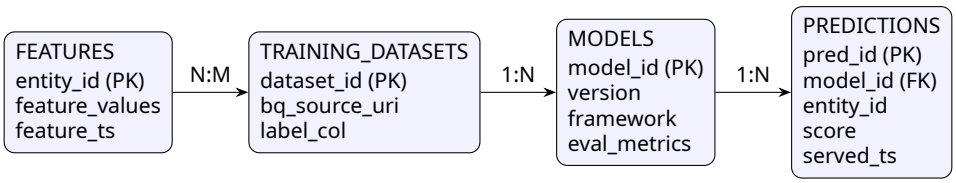

Figure 10: Capstone data model: MLOps lineage from features through registered models to served predictions.

6 Milestone 6: Security, Governance & Exam Prep (Month 6)

Objective: Secure the perimeter, implement Data Mesh, and certify.
Learning outcomes: enforce VPC Service Controls perimeters, govern a Dataplex data mesh, automate DLP de-identification, provision data infrastructure with Terraform and ship Dataform pipelines through CI/CD

Week 21: Security, IAM & Networking

Outcomes: configure VPC Service Controls perimeters and access levels, apply least-privilege custom roles and Workload Identity, choose CMEK versus CSEK, validate the perimeter with a negative test

- **Monday:** VPC Service Controls (Perimeters) and Private Service Connect.
- **Tuesday:** IAM Deep Dive: Custom Roles, Service Accounts, and Workload Identity.
- **Wednesday:** Cloud KMS (CMEK vs. CSEK) in BigQuery and GCS.
- **Thursday:** **Hands-on Project:** Create a VPC Service Perimeter around a BigQuery dataset. Attempt to query it from an external IP to verify the block. Configure an access level to allow your IP.
- **Friday:** **Use Case & Architecture:** Design a highly secure data perimeter for a healthcare provider ensuring no data exfiltration can occur even if IAM credentials are compromised.

Production Note: *Apply least-privilege custom roles and prefer Workload Identity over service-account keys; enable Cloud Audit Logs on all data services and validate the VPC SC perimeter with a negative test (query attempt from outside the perimeter) in every change window.*

Resources: VPC Service Controls docs (https://cloud.google.com/vpc-service-controls/docs); Data Engineering Zoomcamp (https://github.com/DataTalksClub/data-engineering-zoomcamp). **Self-check:**

- What does a VPC SC perimeter block that IAM cannot?
- Why prefer Workload Identity over service-account keys?
- CMEK versus CSEK: who holds the key material in each?
- How do access levels modify perimeter behavior?
- Why test a perimeter with a negative (denied) query?

Week 22: Data Governance & Data Mesh (Dataplex)

Outcomes: organize Dataplex lakes, zones, and assets, tag sensitive columns with policy tags, run AutoDQ and profiling scans, trace lineage from a broken dashboard to its source

- **Monday:** Dataplex architecture: Lakes, Zones (Raw/Curated), and Assets.
- **Tuesday:** Data Catalog: Tag templates, Search, and Data Lineage.
- **Wednesday:** Auto Data Quality (AutoDQ) and Data Profiling.
- **Thursday:** **Hands-on Project:** Set up a Dataplex Lake. Map a GCS bucket and BigQuery dataset as assets. Create a tag template and apply policy tags to restrict access to a 'Salary' column.

- **Friday:** **Use Case & Architecture:** Architect a Data Mesh for a global conglomerate with 5 independent domains, ensuring centralized governance but decentralized data ownership.

Resources: Dataplex docs (https://cloud.google.com/dataplex/docs); DataHub repo (https://github.com/datahub-project/datahub). **Self-check:**

- What distinguishes a raw zone from a curated zone in Dataplex?
- What does a tag template attach to catalog entries?
- What does AutoDQ check without you writing rules?
- In a data mesh, who owns the data product — central team or domain?
- How does lineage help when a downstream dashboard breaks?

Week 23: Compliance & Data Loss Prevention (DLP)

Outcomes: detect PII with InfoTypes and scheduled scans, apply masking and KMS-backed tokenization, propagate GDPR erasure via crypto-shredding, reconcile retention policies with legal holds

- **Monday:** Cloud DLP concepts: InfoTypes, Inspection, and De-identification.
- **Tuesday:** Masking, Tokenization, and Crypto-based hashing.
- **Wednesday:** Automated DLP scanning in BigQuery and GCS. GDPR right-to-erasure propagation: a single deletion request must reach raw GCS lake zones (through object versioning and retention policies), curated BigQuery tables (plus the time-travel window and Fail-safe), Pub/Sub topics (message retention and snapshots), downstream derived tables, caches, and BigQuery backups. Practical patterns: crypto-shredding with per-subject encryption keys in Cloud KMS — destroying the key renders every copy, including backups, unrecoverable — versus row-level deletes that require downstream replay across each layer. Legal-hold and audit-trail exceptions suspend erasure and must be documented.
- **Thursday:** **Hands-on Project:** Write a Python script using the DLP API to scan a block of text, identify emails and credit card numbers, and replace them with asterisks (masking). Erasure drill: encrypt a test subject's PII column with a per-subject Cloud KMS key, then schedule key destruction and verify that both the BigQuery table and a restored backup become unreadable, while rows for other subjects decrypt normally.
- **Friday:** **Use Case & Architecture:** Design a pipeline that ingests third-party customer data, automatically identifies PII, tokenizes it using a KMS key, and stores the safe version in the data warehouse. Extend it with an end-to-end erasure-propagation map: for each storage layer (GCS buckets with versioning/retention, BigQuery tables with time travel, Pub/Sub subscriptions with snapshots, BigQuery backups) document how a crypto-shredding key deletion propagates, and how a legal hold pauses it.

> **Production Note:** *BigQuery time travel, Fail-safe, Pub/Sub snapshots, and backups make a naive* `DELETE` *insufficient for GDPR erasure: deleted rows remain queryable via* `FOR SYSTEM_TIME AS OF` *during the time-travel window, survive seven more days in Fail-safe, and persist in backups and snapshots beyond that. Crypto-shredding erases every copy at once — schedule key destruction in Cloud KMS (keeping the 24-hour destruction delay as a safety net) instead of chasing deletes across zones, and record which legal holds exempt retained copies.*

Resources: Cloud DLP docs (https://cloud.google.com/sensitive-data-protection/docs);

Great Expectations repo (https://github.com/great-expectations/great_expectations). **Self-check:**

- What is an InfoType, and name two built-in examples.
- Masking versus tokenization: which is reversible, and by whom?
- Why use a KMS-wrapped key for tokenization?
- What does scheduled DLP scanning of BigQuery tables detect?
- How do you verify de-identified data is actually unreadable?

Week 24: Production DataOps (IaC, Dataform & CI/CD) & Certification Prep

Outcomes: provision data infrastructure with Terraform, ship Dataform SQLX models and assertions through CI/CD, execute a zero-downtime expand-contract migration, map weak domains from the exam guide

- **Monday:** Infrastructure as Code with the Terraform google provider: GCS buckets, BigQuery datasets, and Composer environments (state files, workspaces, remote backends).
- **Tuesday:** Dataform — BigQuery's native SQL pipeline tool: SQLX models, assertions as data-quality tests, dependency trees, and scheduled runs; note dbt as the cross-platform alternative.
- **Wednesday:** CI/CD for data pipelines with Cloud Build or GitHub Actions: terraform plan/apply on merge, Dataform compilation and assertion runs on every pull request. Zero-downtime schema evolution (expand → migrate → contract): add a nullable column, dual-write and backfill it, switch consumers to a backward-compatible view shim, then drop the old column — each stage is a separate ordered deployment through the same Cloud Build pipeline, coordinated with the Dataform (or dbt) release cycle.
- **Thursday:** **Hands-on Project:** Create a GitHub repository containing Terraform code that provisions a BigQuery dataset and GCS bucket, plus a Dataform project with 3 SQLX models and 5 assertions. Add a Cloud Build trigger that compiles Dataform and runs assertions on every pull request. Expand-contract drill: rename a column in one SQLX model using the four-stage pattern (expand, dual-write/backfill, view shim, contract) across four separate pull requests, verifying the dashboard stays green at every stage.

```
terraform {
  required_providers {
    google = { source = "hashicorp/google", version = "~> 5.0" }
  }
}

provider "google" {
  project = var.project_id
  region  = "us-central1"
}

resource "google_bigquery_dataset" "analytics" {
  dataset_id    = "analytics_curated"
  location      = "US"
  description   = "Curated data products, managed by Terraform"
  default_table_expiration_ms = 7776000000  # 90 days
  labels        = { env = "prod", team = "data-engineering" }
}
```

- **Friday:** Compressed Professional Data Engineer prep: review the exam guide domains against the EHR Healthcare (compliance & migration) and Helium Sports (real-time analytics) case studies, identify weak areas, and schedule both full-length mock exams for the weekend following the program.

Production Note: *Never commit service-account keys to the repository — use Workload Identity Federation for Cloud Build/GitHub Actions; gate terraform apply behind PR approval, and set budget alerts on the CI project as a cost guardrail.*

Production Note: *Failure story: a team dropped the legacy `email` column and renamed it `email_address` in the same release as the Dataform model change; the Looker dashboard deploy followed an hour later and every tile errored on the missing column, paging the on-call. Expand-contract would have kept both columns alive until consumers switched. Deploy-order rule: schema expands first, consumers migrate to the view shim, contraction ships last — never bundle all three in one release.*

Resources: Terraform google provider docs (https://registry.terraform.io/providers/hashicorp/google/latest/docs); Dataform docs (https://cloud.google.com/dataform/docs).

Self-check:

- Why must Terraform state live in a remote backend for team work?
- What does a Dataform assertion guarantee about a model's output?
- What should run on every pull request: plan, apply, or both?
- Why is Workload Identity Federation safer than committed keys?
- How do SQLX dependencies determine Dataform's execution order?

CAPSTONE PROJECT - Milestone 6: Secure, Governed Data Mesh. Build a multi-project architecture. Project A ingests streaming PII data and uses DLP to tokenize it. Project B (Analytics) hosts BigLake tables. Project C (Governance) uses Dataplex to enforce row-level security and VPC Service Controls to lock down all API access.

Acceptance Criteria:

- End-to-end mesh deploys from a clean environment following the repo README (Terraform or documented gcloud scripts for all three projects).
- Architecture diagram committed showing projects, VPC SC perimeter, and governance flows.
- At least 3 automated data-quality tests (Dataplex AutoDQ scans or Dataform assertions) with a failing-case demonstration.
- Monitoring/alerting evidence: audit-log-based alert for perimeter violations and DLP findings (screenshot or exported config).
- Monthly cost estimate with 2 optimization decisions documented (DLP scan sampling, Dataplex tier, per-project budgets).
- Security review: least-privilege roles per project, no hardcoded secrets, tokenized PII verified unreadable in the clear, negative perimeter test passed.

Milestone 6 Capstone: Reference Solution Sketch

- **Architecture:** three projects — A (ingest + DLP tokenization), B (BigLake analytics), C (Dataplex governance) — inside one VPC Service Controls perimeter, everything provisioned by Terraform — blast radius of any single compromise is one project.
- **Steps:** (1) Terraform the three projects and perimeter; (2) stream PII into Project A and tokenize with DLP + Cloud KMS; (3) expose only tokenized tables as BigLake assets in Project B; (4) enforce row-level security and policy tags from Project C; (5) run the negative perimeter test and audit-log alert.
- **Hardest part — deterministic, key-managed tokenization:**

```
from google.cloud import dlp_v2
```

```
dlp = dlp_v2.DlpServiceClient()
crypto = {"crypto_key": {"kms_wrapped": {
            "wrapped_key": wrapped_key_bytes,
            "crypto_key_name": kms_key_resource}},
          "surrogate_info_type": {"name": "USER_TOKEN"}}
# deterministic FFX: same input -> same token, joins still work,
# but detokenization requires an auditable Cloud KMS permission
```

- **Pitfall:** the perimeter blocks Composer/Build API calls from outside — add every CI and orchestration service to the perimeter or an ingress rule.
- **Pitfall:** cross-project Terraform applies race — order them with explicit `depends_on` or separate state per project.
- **Pitfall:** tokenized PII can still be re-identified through quasi-identifiers (zip + birthdate) — run DLP risk analysis on the ”safe” tables.

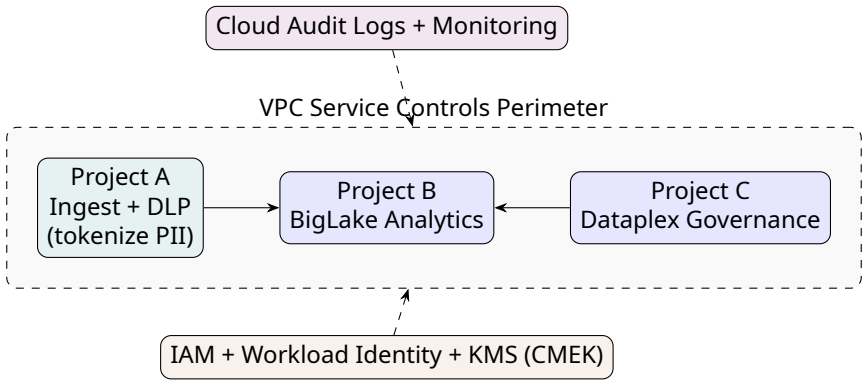

Figure 11: Milestone 6 reference architecture: multi-project security mesh behind VPC Service Controls.

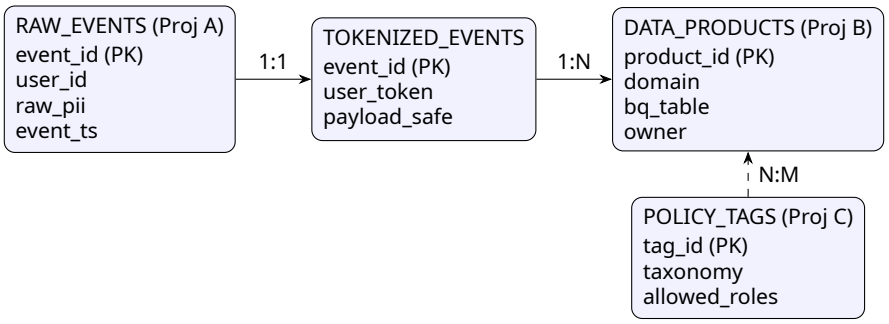

Figure 12: Capstone data model: secure data mesh with tokenized PII and centrally governed policy tags.

7 Appendix A: Glossary

Slot BigQuery's unit of compute capacity; queries consume slots from on-demand or edition-based reservations.

BigLake
BigQuery storage engine that exposes GCS files as tables with fine-grained (row/column) security.

Analytics Hub
BigQuery service for governed data sharing and data clean rooms across organizations.

BI Engine
In-memory acceleration layer for BigQuery that speeds up dashboard queries.

Dataflow
Managed runner for Apache Beam batch and streaming pipelines with autoscaling.

Watermark
Beam's estimate of event-time completeness; data arriving behind it is considered late.

Allowed Lateness
Windowing parameter controlling how long a window accepts late-arriving events.

Pub/Sub Dead-Letter Topic
Topic that receives messages a subscription failed to deliver after N attempts.

Spanner TrueTime
Globally synchronized clock API enabling Spanner's external consistency guarantees.

Bigtable Row Key
Primary index of a Bigtable table; poor key design causes hotspotting.

Dataproc
Managed Hadoop/Spark service; supports ephemeral and serverless clusters.

Datastream
Serverless change data capture (CDC) service replicating database changes into GCS or BigQuery.

Cloud Composer
Managed Apache Airflow environment (v2 runs on GKE with autoscaling).

DAG
Directed Acyclic Graph — an Airflow pipeline definition of tasks and dependencies.

Dataform
BigQuery-native tool for SQL pipelines: SQLX models, assertions, and scheduled runs.

SQLX
Dataform's SQL extension adding model configs, dependencies, and assertions to plain SQL.

Assertion
Declarative data-quality test in Dataform (or dbt) that fails a run when rows violate a condition.

Dataplex
Governance fabric organizing lakes, zones, and assets with Auto Data Quality and lineage.

VPC Service Controls
Perimeter-based security that blocks data exfiltration even with valid IAM credentials.

CMEK
Customer-Managed Encryption Keys in Cloud KMS, giving you key rotation and revocation control.

DLP InfoType
A category of sensitive data (e.g., email, credit card) detected by Cloud DLP.

Vertex AI Feature Store
Managed store for ML features with online and offline serving.

Model Drift
Degradation of model performance when production data diverges from training data.

LookML
Looker's semantic modeling language defining dimensions, measures, and explores.

8 Appendix B: Assessment & Grading Model

Each of the six capstones is graded on the following weighted rubric:

Category	Weight	What is evaluated
Correctness	30%	Pipeline runs end-to-end from a clean environment and produces correct results.
Reliability & tests	20%	Automated data-quality tests, idempotent re-runs, failure handling (DLQ/retries).
Security	15%	Least-privilege roles, no hardcoded secrets, PII handling, perimeter checks.
Cost awareness	15%	Documented monthly estimate with at least 2 justified optimization decisions.
Documentation	10%	README, architecture diagram, and runbook quality.
Demo	10%	Live or recorded walkthrough explaining design trade-offs.

The final readiness score combines the average capstone grade (70%) with the two Professional Data Engineer mock exams from Week 24 (30%). A mock-exam score below 70% in any domain triggers a targeted one-week review loop before scheduling the real exam.

9 Appendix C: Interview Preparation

1. How would you design globally consistent transactions for a payment ledger? *(Week 3)*
2. Explain BigQuery partitioning vs. clustering and when to combine them. *(Week 6)*
3. How do you share data with a partner without exposing raw PII? *(Weeks 8, 23)*
4. Walk through migrating an on-premise Hadoop cluster to GCP. *(Week 9)*
5. How does CDC replication into BigQuery work, and how do you monitor its freshness? *(Week 11)*
6. Design an idempotent, retry-safe orchestration layer for 15 dependent systems. *(Week 12)*
7. How do you achieve exactly-once semantics in a Pub/Sub to Dataflow pipeline? *(Weeks 13–14)*
8. Explain event time, watermarks, and how you handle very late mobile data. *(Week 15)*
9. Describe a full MLOps lifecycle with drift detection and automated retraining. *(Week 19)*
10. How do VPC Service Controls and Dataplex combine to secure a data mesh? *(Weeks 21–22)*

10 Appendix D: Self-Check Answer Key

Week 1:

1. Archive — lowest storage price for rarely read data.
2. Only the storage class (access cost/latency), never the data itself.
3. Petabyte-scale transfers where bandwidth makes online transfer impractical.
4. Copy the object's older generation back, e.g. `gs://bucket/object#generation`.
5. Deletion, corruption, and regional loss still need replication plus tested recovery.

Week 2:

1. A synchronous standby in another zone with automatic failover.
2. PITR replays WAL to a timestamp; in-place restore would destroy current data.
3. When traffic must stay inside the VPC and off the public internet.
4. Analytical scans and aggregations over large PostgreSQL datasets.
5. Continuous CDC replication followed by a short cutover window.

Week 3:

1. Globally agreed commit timestamps, enabling external (strong) consistency.
2. Monotonic keys concentrate writes on one split, causing hotspots.
3. Co-locating child rows with parents for fast local joins and cascades.
4. Synchronous multi-region replication coordinated by TrueTime.
5. Small or single-region workloads — Spanner's cost and complexity dominate.

Week 4:

1. Sequential keys funnel writes onto a single tablet server.
2. Spreads load across tablets; sacrifices efficient range scans.
3. Routing and replication policy per application.
4. A heatmap of row-key access that exposes hotspots.
5. How many versions or how much cell history to retain.

Week 5:

1. Colossus (storage), Jupiter (network), Dremel (compute) scale independently.
2. Steady, high-utilization workloads needing predictable monthly spend.
3. Exactly what one row represents; every fact must match that grain.
4. When history matters; implemented with valid_from, valid_to, is_current.
5. A business key has many historical rows; only one is current.

Week 6:

1. Partitioning prunes whole partitions; clustering sorts within them for finer pruning.
2. High-selectivity filters on columns that are not the partition key.
3. It precomputes query results at the cost of storage and refresh compute.
4. One worker processing far more rows or time than its peers in a stage.
5. Repeated dashboard queries; wasted on one-off ad-hoc scans.

Week 7:

1. External tables read files in place; native tables use BigQuery-managed storage.
2. `ST_WITHIN` (or `ST_CONTAINS` with arguments reversed).
3. Source-cloud egress charges plus cross-region latency.
4. BigQuery cannot reorganize data it does not store.
5. Small or fast-changing data where ingestion overhead is unjustified.

Week 8:

1. It applies BigQuery row/column security to files in GCS.
2. Metadata over unstructured files such as PDFs and images.
3. Linked datasets: subscribers query in place with no copy.
4. Each party's raw rows and PII from the other.
5. Central, auditable masking without maintaining per-role views.

Week 9:

1. You pay only for job runtime instead of an idle cluster.
2. Cheap but reclaimable — use for workers and keep jobs idempotent.
3. Decouples storage from compute so each scales (and bills) independently.
4. Executor memory undersized for skewed partitions.
5. Pending YARN memory/containers beyond current capacity.

Week 10:

1. CDAP (Cask Data Application Platform).
2. Interactive cleaning on samples before scaled production runs.
3. Analysts building pipelines without writing Java or Python.
4. Sources and sinks reachable only inside the VPC.
5. Runtime arguments such as dates, paths, and table names.

Week 11:

1. By reading the source database's replication/write-ahead log.
2. Renaming a column; adding a nullable column is non-breaking.
3. Rows with unknown or mismatched fields, unless schema updates are relaxed.
4. The producer owns it; consumers version and agree to changes.
5. How far the destination lags the source.

Week 12:

1. Retries re-execute tasks; side effects must be safely repeatable.
2. The notification runs whether upstream tasks succeed or fail.
3. An autoscaling GKE cluster.
4. When the run depends on an external file's arrival, not a clock.
5. Only small metadata — never large payloads.

Week 13:

1. It is forwarded to the dead-letter topic.
2. The broker rejects publishes that violate the Avro/proto schema.
3. Retry-induced duplicates; deduplicate on message IDs downstream.
4. Very high, steady throughput with zonal (not regional) tolerance.
5. Consumers are stuck or undersized and the backlog is growing.

Week 14:

1. A PTransform is an operation; a ParDo is an element-wise user function.
2. Logic bugs are caught cheaply before paying for runner deployment.
3. Shuffle and state handling, enabling faster autoscaling.
4. When you reuse a parameterized pipeline across teams or jobs.
5. It adds workers based on backlog and CPU utilization.

Week 15:

1. Event-time progress; events older than it are considered late.
2. State grows without bound, exhausting worker memory/disk and defeating autoscaling; timers fire as the watermark passes, so garbage collection frees each key's state after its TTL.
3. `SEEN` stores already-emitted event IDs per key; `EXPIRY` fires at the TTL to clear that bag.
4. Both inputs must share the same window assignment and join key, with outputs gated on watermark alignment; unmatched elements are buffered per window until allowed lateness expires.
5. Streaming Engine deduplicates shuffle records and the Storage Write API commits offsets atomically; at-least-once plus downstream dedup is cheaper when sinks lack exactly-once support or duplicates are tolerable.

Week 16:

1. Metrics are defined once in LookML and reused by every explore.
2. It activates warehouse data inside operational tools, not just reports.
3. The same customer syncs as duplicate or mismatched records.

4. Freshness versus API rate limits and per-call cost.
5. Governance, row-level security, and embedded analytics at scale.

Week 17:

1. Logistic regression for binary classification.
2. Automatic seasonality, holidays, and hyperparameter choices.
3. Precision, recall, and ROC AUC — never accuracy alone.
4. No data movement; governance and security stay inside BigQuery.
5. A wide gap between training and evaluation metrics.

Week 18:

1. When data is tabular and custom architecture gains little.
2. Model versions, lineage, and deployment targets.
3. Online prediction serves the low-latency checkout flow.
4. Which input features most drove a given prediction.
5. Endpoints add managed scaling, authentication, and traffic splitting.

Week 19:

1. Keeps training and serving features consistent, preventing skew.
2. Online serving — low-latency feature lookups per request.
3. A containerized, reusable step with a declared input/output contract.
4. Drift plus data-quality checks passing an evaluation gate.
5. Skew compares serving inputs to training; drift tracks input shift over time.

Week 20:

1. When a pretrained API solves the task without a custom model.
2. Retries with backoff, batching, and quota alerts.
3. The Natural Language API.
4. Store extracted text and labels in BigQuery with a search index.
5. Per-call pricing scales directly with message volume.

Week 21:

1. Exfiltration by valid credentials to destinations outside the perimeter.
2. No exportable key material exists to be leaked.
3. CMEK lives in Cloud KMS; with CSEK you supply the key yourself.
4. They let specified IPs or identities through the perimeter.
5. A deny result proves the perimeter actually enforces.

Week 22:

1. Raw zones hold unprocessed landings; curated zones are governed and quality-checked.
2. Structured metadata such as PII flags, owner, and quality scores.
3. Nulls, ranges, freshness, and distribution anomalies.
4. The domain owns the product; the center sets guardrails.
5. It traces the broken column back to the upstream pipeline.

Week 23:

1. A category of sensitive data, e.g. EMAIL_ADDRESS, CREDIT_CARD_NUMBER.
2. Tokenization — reversible only by holders of the key.
3. Detokenization then requires auditable KMS permissions.
4. New or previously undetected PII appearing over time.
5. Attempt to read raw values and verify the denial (negative test).

Week 24:

1. Remote state gives locking and survives local machines.
2. Rows violating its condition fail the pipeline run.
3. `terraform plan` plus Dataform compile/assert; apply only on merge.
4. Short-lived federated tokens replace committed key files.
5. `ref()` dependencies build the DAG that orders execution.

11 Appendix E: Datasets & Project Data

Public datasets (BigQuery-hosted, no ingestion cost to store)

- **NYC Taxi trips** — `bigquery-public-data.new_york_taxi_trips`: partitioned-by-design trip records; used in Week 5–6 SQL optimization labs and the Milestone 2 lakehouse capstone.
- **StackOverflow** — `bigquery-public-data.stackoverflow`: posts, users, and badges; used for Week 5 array/struct and window-function labs.
- **GHCN weather** — `bigquery-public-data.ghcn_d`: daily global weather observations; used in Week 7 GIS/federated labs and BQML forecasting in Week 17.

Capstone datasets

- **Olist Brazilian e-commerce** (Kaggle: https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce) — orders, items, customers, payments, reviews; primary dataset for the Milestone 1 multi-tier backend and Milestone 2 star-schema capstones.
- **Simulated card transactions** — generated by the Week 13 Pub/Sub publisher script; feeds the Milestone 4 fraud-detection capstone.
- **Google Analytics sample** — `bigquery-public-data.google_analytics_sample`; used for Week 17 BQML churn modeling.

Synthetic data generation

Use the Python `Faker` library for names, addresses, and transactions (e.g. `pip install faker`); expected volumes: Milestone 1 $\approx$ 10K rows per store, Milestone 2 $\approx$ 5 years / 10M fact rows, Milestones 3–4 $\approx$ 1K–5K streaming events per lab run, Milestone 5 $\approx$ 100K labeled training rows. Keep generators deterministic (seeded) so data-quality tests are reproducible.

12 Appendix F: Portfolio & Presentation Guide

GitHub README structure (every capstone)

1. **Problem** — one paragraph: who has the pain and what decision the pipeline supports.
2. **Architecture diagram first** — the TikZ/draw.io figure from this syllabus, before any code.
3. **Setup** — exact commands to rebuild from a clean project (as the rubric requires).
4. **Screenshots** — dashboard, DAG run, monitoring alert firing.
5. **Cost estimate** — monthly table with your 2 documented optimization decisions.
6. **Lessons learned** — 3 bullets: what broke, what you would change.

Resume bullet templates (two per milestone)

- **M1:** "Built a multi-tier GCP data backend (Cloud SQL, Spanner, Bigtable, GCS) with VPC-private connectivity and automated failover tests." / "Cut storage spend X% via lifecycle policies across 4 storage classes."

- **M2:** "Designed a governed BigQuery lakehouse with row/column-level security and materialized views serving executive dashboards." / "Reduced query spend X% by migrating on-demand workloads to slot Editions with partitioning and clustering."

- **M3:** "Automated a batch ELT pipeline with Cloud Composer, Datastream CDC, and serverless Dataproc with end-to-end data-quality gates." / "Achieved zero double-loads by making all Spark tasks idempotent under preemptible-VM retries."

- **M4:** "Built a real-time fraud-detection pipeline (Pub/Sub, Dataflow sliding windows, BigQuery, Looker) with DLQ and watermark-lag alerting." / "Implemented exactly-once semantics and late-data handling for mobile telemetry at X events/sec."

- **M5:** "Shipped an end-to-end MLOps pipeline (BQML, Vertex AI, Composer) with drift monitoring and gated retraining." / "Deployed an AutoML churn model to a Vertex endpoint with explainability and evaluation thresholds."

- **M6:** "Provisioned a 3-project secure data mesh (VPC SC, Dataplex, DLP tokenization) entirely with Terraform and CI/CD." / "Passed the Professional Data Engineer practice domains with 80%+ after a self-built 24-week program."

Talking about your project in interviews

Lead with the problem and the constraint (cost, latency, compliance), not the service list. Walk the interviewer through the architecture diagram top to bottom, then volunteer one trade-off you made and why (e.g. on-demand vs. slots, push vs. pull subscriptions, SCD1 vs. SCD2). Finish with a failure story — a DLQ that filled, a watermark that lagged, a PITR drill that caught a bad delete — and what you changed. Interviewers hire engineers who have operated systems, not just built demos.

13 6-Month Progress Tracking Timetable

Week	Primary Focus	Key Project / Capstone	Status
1	GCS & Transfers	GCS Lifecycle & Transfer Job	☐
2	Cloud SQL & AlloyDB	HA PostgreSQL Failover Simulation	☐
3	Cloud Spanner	Global Interleaved Schema	☐
4	Cloud Bigtable	**Cap 1: E-Commerce Multi-Tier Data**	☐
5	BigQuery Architecture	Advanced SQL Window Functions	☐
6	BQ Optimization	Partitioning & Clustering Benchmarks	☐
7	BQ Advanced	GIS & External Tables	☐
8	Lakehouse & Analytics Hub	**Cap 2: Enterprise Data Lakehouse**	☐
9	Cloud Dataproc	Spark Job on Ephemeral Cluster	☐
10	Cloud Data Fusion	No-code Wrangler ETL Pipeline	☐
11	Datastream (CDC)	PostgreSQL to BQ Logical Sync	☐
12	Cloud Composer	**Cap 3: Automated Batch ELT**	☐
13	Cloud Pub/Sub	DLQ and Push/Pull Scripting	☐
14	Dataflow Basics	Beam WordCount on Cloud Runner	☐
15	Dataflow Advanced	Session Windows & Late Data logic	☐
16	Looker & BI	**Cap 4: Real-Time Fraud Telemetry**	☐
17	BigQuery ML	Churn Prediction & Evaluation	☐
18	Vertex AI Foundations	AutoML Deployment & cURL Predict	☐
19	MLOps & Pipelines	Kubeflow Pipeline Orchestration	☐
20	AI APIs	**Cap 5: End-to-End MLOps Pipeline**	☐
21	IAM & VPC SC	Service Perimeters & Access Levels	☐
22	Dataplex & Data Mesh	AutoDQ and Policy Tags	☐
23	Cloud DLP	Python Tokenization/Masking Script	☐
24	DataOps & Exam Prep	**Cap 6: Secure Data Mesh** + Terraform/Dataform CI/CD Repo	☐

"Consistency is the architecture of mastery."

Best of luck on your 6-month journey to becoming a Master Google Cloud Data Engineer!